VEB DASTURLAR XAVFSIZLIGI

A12. Veb ilovalar xavfsizligini kompleks tahlil qilish va himoyalash usullari

1-VARIANT — “Talaba qaydlari” ilovasining yakuniy xavfsizlik auditi

Mashg‘ulot maqsadi. Veb ilovani belgilangan chegarada tahlil qilish, zaiflikni qayta hosil qilish, uning sababini koddan topish, xavfni ustuvorlashtirish va himoyadan keyingi qayta sinov bilan xulosani asoslash. Oldingi mashg‘ulotlardagi alohida tekshiruvlar yagona audit jarayoniga birlashtiriladi.

Kutiladigan natija. Talaba tekshiruv doirasini yozadi; IDOR, SQL inyeksiya va reflected XSSni farqlaydi; xavfsizlik sarlavhalarini baholaydi; “topildi — tuzatildi — qayta tekshirildi” dalillarini bog‘laydi. Qolgan xavflar va tekshirilmagan yo‘nalishlarni ko‘rsatib, chiqarishga ruxsat berish yoki bermaslik qarorini asoslaydi.

Ko‘rsatkich	Mashg‘ulot sharti
Fan / semestr	Veb dasturlar xavfsizligi / faqat 5-semestr
Mashg‘ulot	A12, 1-variant; individual yakuniy amaliy ish
Auditoriya	30 ta kompyuter, lokal tarmoq va internet
Kompyuter	Windows 11; 8 yadroli CPU; RAM 32 GB; SSD 1 TB
Virtual muhit	VMware Workstation yoki VirtualBox; Ubuntu 24.04 LTS
VM resursi	2 vCPU, 4 GB RAM, 25 GB disk; NAT adapter
Davomiylik / baho	Tayyor muhitda 80 daqiqa / 10 ball

Vositalar va darsdan oldingi tayyorgarlik

1. Ubuntu Desktop VM, Python 3, curl, nano va Firefox yoki Chromium. Python standart kutubxonasi yetarli; pip paketlari va pullik skaner kerak emas.
2. O‘qituvchi 1-ilovadagi app.py va 2-ilovadagi audit.py fayllarini oldindan tarqatadi. VM va paketlarni o‘rnatish, Worddan kodlarni ko‘chirish 80 daqiqalik asosiy ishga kiritilmaydi.

Ish chegarasi. Faqat o‘z VMingizdagi 127.0.0.1:8012 xizmatini tekshiring. Ma’lumotlar sun’iy. Ilova zaif rejimni ataylab o‘z ichiga oladi; umumiy LAN yoki internetga ochilmaydi. Bitta kompyuterdagi tajriba boshqa 29 ta kompyuter ishiga ta’sir qilmaydi.

1.1. Audit doirasi va ishonch chegaralari

Ssenariy. “Talaba qaydlari” xizmatida Alice va Bobning shaxsiy qaydlari bor. Har foydalanuvchi faqat o‘z qaydini o‘qishi va qidirishi kerak. Preview sahifasi esa mehmonlarga ochiq bo‘lib, kiritilgan matnni bajariladigan kodga aylantirmasdan ko‘rsatishi kerak.

Obyekt / yo‘l	Kirish sharti	Kutiladigan xatti-harakat
/health	Ochiq	200; faol secure rejimini ko‘rsatadi
/api/note?id=…	Demo token	Faqat egasining bitta qaydi
/api/search?q=…	Demo token	Faqat egasining mos qaydlari
/preview?text=…	Ochiq	Matn HTML matn kontekstida chiqariladi

ID	Egasi	Sun’iy mazmun
1	alice	Alice plan
2	bob	Bob private
3	alice	Alice report

curl va brauzer HTTP orqali ilovaga murojaat qiladi; ilova xotiradagi SQLite bazasini o‘qiydi. Birinchi ishonch chegarasi — mijozdan serverga kelgan parametrlar. Ikkinchisi — server foydalanuvchisi bilan ma’lumot egasi o‘rtasidagi ruxsat. Uchinchi chegara — server javobidagi matnning brauzerda HTML sifatida talqin qilinishi.

Tekshiruvdan oldin yozib olinadigan shartlar

3. Nishon: 127.0.0.1:8012; bajaruvchi: talabaning F.I.Sh.; sana; VM va Python versiyasi; boshlang‘ich rejim secure=False.
4. Kirish: X-Lab-Token: demo-alice va demo-bob. Ular login jarayonini almashtiruvchi ochiq o‘quv belgilaridir; haqiqiy autentifikatsiya yoki sir saqlash yechimi emas.
5. Ruxsat etilgan ish: berilgan oz sonli GET so‘rovlari, kod tahlili, himoyalangan rejimda qayta sinov. Bazani o‘chirish, katta trafik va tashqi nishonlar ushbu ishga kirmaydi.

Dalil D1. Endpointlar, aktivlar, foydalanuvchi–obyekt ruxsatlari va doira. Kompleks audit barcha turdagi hujumlarni sinash degani emas: tanlangan xavflar chuqur tekshiriladi, qolgan yo‘nalishlar alohida “tekshirilmagan” deb qayd etiladi.

OWASP Top 10:2025 xavflarni guruhlash uchun ishlatiladi. ASVS 5.0.0 esa batafsil tekshiruv talablarini tanlashga yordam beradi. Mazkur kichik ish butun Top 10 yoki ASVSga muvofiqlik sertifikati hisoblanmaydi. [1, 2]

1.2. Muhitni tayyorlash va xizmatni boshlash

1-topshiriq. Tayyor VM bo‘lmasa, Ubuntu 24.04 LTS Desktop ISO orqali 2 vCPU, 4 GB RAM, 25 GB disk va NAT adapterli VM yarating. OTni o‘rnating, oddiy foydalanuvchi bilan kiring. Brauzer ham shu VM ichida ishlatiladi. Quyidagi buyruqlar Ubuntu Bash uchun:

```
sudo apt update
sudo apt install -y python3 curl nano firefox
umask 077
mkdir -p ~/a12-lab
cd ~/a12-lab
```

1-ilovaning A, B va C qismlarini tartib bilan bitta app.py fayliga, 2-ilovaning A va B qismlarini bitta audit.py fayliga saqlang. Fayllar UTF-8 bo‘lsin; Python surilishlari va oddiy qo‘shtirnoqlar o‘zgarmasin.

```
python3 -m py_compile app.py audit.py
python3 --version
curl --version
sha256sum app.py audit.py > source.sha256
python3 -u app.py > before-server.log 2>&1
```

1-terminal xizmat ishlab turgani uchun band qoladi. Dastur sudo bilan ishga tushirilmaydi. Dars uchun yangi a12-lab katalogidan foydalaning; eski tajriba bo‘lsa, oldingi fayllarni avval alohida katalogga saqlang. “>” mavjud logni almashtiradi.

2-terminalda boshlang‘ich nazorat

```
cd ~/a12-lab
U=http://127.0.0.1:8012
ss -ltn | grep ":8012"
curl --noproxy "*" -i "$U/health"
curl --noproxy "*" -i "$U/api/note?id=1"
```

Tinglovchi 127.0.0.1:8012 bo‘lsin. Health: 200, ok=true, secure=false. Tokensiz note so‘rovi: 401 va unauthorized. Bu ikki natija xizmat ishlashini hamda API uchun demo identifikator tekshirilishini ko‘rsatadi.

```
curl --noproxy "*" -i "$U/api/note?id=1" \
  -H "X-Lab-Token: demo-alice"
```

Natija 200, id=1, owner=alice, title=Alice plan. X-Request-ID qiymatini yozib oling; shu identifikator keyin server jurnalida topiladi. Cache-Control: no-store ikkala rejimda mavjud.

Dalil D2. VM va dastur versiyalari, port, secure=false, 401 va qonuniy 200 javoblari. NAT faqat o‘rnatish uchun internet beradi; tajriba trafigi VMning loopback manzilida qoladi.

1.3. Obyektga kirish nazoratini tekshirish

2-topshiriq. Alice nomidan faqat id qiymatini 1 dan 2 ga almashtiring. Token bir xil qoladi:

```
curl --noproxy "*" -s -i "$U/api/note?id=2" \
  -H "X-Lab-Token: demo-alice" | tee f01-before.txt
curl --noproxy "*" -i "$U/api/note?id=2" \
  -H "X-Lab-Token: demo-bob"
```

Zaif rejimda ikkala so‘rov ham 200 va Bob private qaydini qaytaradi. Bob uchun bu qonuniy; Alice uchun esa ruxsatsiz o‘qish. Bu IDOR ko‘rinishidagi obyekt darajasidagi avtorizatsiya xatosi. Faqat taxmin qilinadigan ID mavjudligi emas, serverning egani tekshirmasligi asosiy sababdir. [3]

Koddan sababni topish

app.py ichidagi get_note funksiyasini ko‘ring. Zaif tarmoqda SQL parametrli bo‘lsa ham, owner sharti yo‘q:

```
return DB.execute('SELECT * FROM notes WHERE id=?',
                  (note_id,)).fetchone()
```

SQL inyeksiyadan himoya va avtorizatsiya turli talablar. Parametrli so‘rov note_idni SQL kodga aylantirmaydi, lekin foydalanuvchining shu obyektni o‘qish huquqini o‘zi tekshirmaydi.

Himoyalangan tarmoqdagi nazorat

```
return DB.execute('SELECT * FROM notes WHERE id=? '
                  'AND owner=?', (note_id, owner)).fetchone()
```

owner mijozning qidiruv parametridan emas, server token xaritasidan olinadi. Shart id va egani birga tekshiradi. Mos yozuv topilmasa umumiy 404 qaytariladi. Ushbu model begona va mavjud bo‘lmagan obyektga bir xil javob beradi; real ilovada 403 yoki 404 tanlovi siyosatga bog‘liq.

So‘rov	Zaif holat	Himoyadan keyin
Alice → id=1	200 / Alice plan	200 / Alice plan
Alice → id=2	200 / Bob private	404 / not_found
Bob → id=2	200 / Bob private	200 / Bob private
Alice → id=999	404	404

Dalil D3 / F01. Bir xil Alice tokeni bilan id=1 va id=2 javoblari, get_note sabab satri va keyingi 404 dalili. Ta’sir — boshqa foydalanuvchi qaydining maxfiyligi buziladi. Faqat 404 ko‘rsatish yetarli emas: o‘z qaydi ishlashi ham tasdiqlanadi.

1.4. Qidiruvdagi SQL inyeksiyani tekshirish

3-topshiriq. Avval odatiy qidiruvni bajaring. --data-urlencode parametrni URL uchun kodlaydi; -G ma’lumotni GET query qismiga joylaydi:

```
curl --noproxy "*" -s -G "$U/api/search" \
  -H "X-Lab-Token: demo-alice" --data-urlencode "q=Alice"
```

200 javob tanasidagi notes ro‘yxati id=1 va id=3 yozuvlardan iborat. Keyin aynan shu endpointga quyidagi kichik sinov kiritmasini yuboring:

```
curl --noproxy "*" -s -i -G "$U/api/search" \
  -H "X-Lab-Token: demo-alice" \
  --data-urlencode "q=' OR 1=1-- " | tee f02-before.txt
```

Zaif holatda notes ichida uchala yozuv, jumladan Bob private paydo bo‘ladi. Qo‘shtirnoq LIKE satrini yopadi, OR 1=1 shartni rost qiladi, -- esa qolgan qismini izohga aylantiradi. Bu SQLite uchun yozilgan nazorat misoli; ma’lumotni o‘zgartirmaydi.

SQL tuzilishidagi o‘zgarishni izohlang

```
SELECT * FROM notes WHERE owner='alice'
AND title LIKE '%' OR 1=1-- %'
```

AND amali ORdan oldin baholanadi. Natijada oldingi cheklov bilan birga “yoki rost” sharti hosil bo‘ladi va egaga oid chegaraning ta’siri yo‘qoladi. F01dan farqi: bu yerda server qidiruv kiritmasini SQL sintaksisiga qo‘shmoqda.

Parametrli tuzatish

```
return DB.execute('SELECT * FROM notes WHERE owner=? '
                  'AND title LIKE ?', (owner, '%' + term + '%'))
```

So‘rov tuzilishi oldindan belgilanadi, term alohida parametr bo‘ladi. Xuddi shu sinov satri endi oddiy qidiriladigan matn sifatida ko‘riladi va notes=[] qaytadi. Apostrofni qora ro‘yxat bilan o‘chirish o‘rniga parametrli so‘rov qo‘llanadi. [4]

Qonuniy qidiruvni qayta tekshirish shart: q=Alice hali ham id=1 va id=3ni qaytarsin. LIKE ichidagi % va _ belgilarining wildcard ma’nosi parametrli so‘rovda ham saqlanadi; ushbu ilovada bu qidiruv funksiyasining xususiyati. Ular boshqa egaga o‘tish huquqini bermasligi kerak.

Dalil D4 / F02. Oddiy qidiruv, uch yozuvli zaif javob, sabab SQLi va himoyadan keyingi bo‘sh ro‘yxat. “So‘rov xato berdi”ning o‘zi SQL inyeksiyani tasdiqlamaydi; bu yerda boshqa egaga tegishli yozuvning chiqishi ta’sir dalilidir.

1.5. XSS va javob sarlavhalarini baholash

4-topshiriq. Zarar yetkazmaydigan skript faqat brauzer varag‘i sarlavhasini A12-XSSga almashtiradi. Tashqi resurs, cookie o‘qish yoki ma’lumot jo‘natish yo‘q. 2-terminalda:

```
P='<script>document.title="A12-XSS"</script>'
curl --noproxy "*" -s -i -G "$U/preview" \
  --data-urlencode "text=$P" | tee f03-before.txt
```

Zaif javobda <script> tegi o‘zgarmay HTMLga qo‘shiladi. curl JavaScriptni bajarmaydi: bajarilishini brauzer bilan alohida tekshiring. Ubuntu VM brauzerining manzil satriga quyidagini qo‘ying:

```
http://127.0.0.1:8012/preview?text=%3Cscript%3E
document.title%3D%22A12-XSS%22%3C%2Fscript%3E
```

Yuqoridagi ikki satrni orasiga bo‘sh joy qo‘shmasdan bitta URL qilib kiriting. Zaif rejimda tab sarlavhasi A12-XSS bo‘ladi. DevTools → Network orqali aynan localhost javobini va uning sarlavhalarini ko‘ring.

Himoya sababini ajrating

```
value = html.escape(text, quote=True) if SECURE else text
```

Himoyalangan rejimda < va > belgilar HTML entityga aylanadi; sahifa skriptni matn sifatida ko‘rsatadi. Bu HTML matn kontekstiga mos kodlash. JavaScript, URL yoki CSS konteksti uchun shu usulni ko‘r-ko‘rona ishlatish mumkin emas. [5]

Nazorat	Zaif rejim	Himoyalangan rejim
Skript natijasi	Tab: A12-XSS	Tab: A12 Preview; skript matn
HTML tanasi	<script>…</script>	<script>…</script>
X-Content-Type-Options	Mavjud emas	nosniff
Content-Security-Policy	Mavjud emas	default-src 'none'; qo‘shimcha cheklovlar

CSPning to‘liq qiymati kodda berilgan: skript va resurslarni yuklash, freymga joylash, base hamda form-action cheklanadi. CSP qo‘shimcha qatlam; XSSning ildiz sababi kodlanmagan chiqarishdir. Sarlavha yetishmasligi F04 sifatida alohida qayd etiladi, lekin F03 ta’siri ikki marta hisoblanmaydi.

Dalil D5 / F03–F04. HTTP javob, brauzer tabining oldin/keyin holati va sarlavhalar. HSTS bu HTTP loopback modelida baholanmaydi; HTTPS/TLS tekshiruvi alohida doiraga kiritiladi.

1.6. Himoyani qo‘llash va regressiya sinovi

5-topshiriq. Himoyadan oldin 2-terminalda 2-ilovadagi tekshiruv to‘plamini ishga tushiring. U faqat belgilangan localhost manziliga murojaat qiladi:

```
python3 audit.py > before.txt 2>&1
BEFORE_RC=$?
cat before.txt
echo "$BEFORE_RC"
```

Kutiladigan natija: SUMMARY 8 / 12; chiqish kodi 1. T05, T07, T08 va T09 — FAIL. Bu xavfsizlik talablari buzilganini bildiradi; qolgan funksional nazoratlar ishlaydi. Ulanish yoki Python xatosi bo‘lsa, SUMMARY olinmaguncha natijani audit deb qabul qilmang.

Tuzatishlarni faollashtirish

get_note, search, preview va reply funksiyalaridagi SECURE tarmoqlarini ko‘rib, har birining qaysi F01–F04ga taalluqli ekanini hisobotga yozing. Tayyor tuzatishlar --secure bilan faollashadi; faqat bayroqni yoqish sabab tahlili o‘rnini bosa olmaydi.

1-terminalda Ctrl+C bosing. Keyin shu terminalda:

```
python3 -u app.py --secure > after-server.log 2>&1
```

2-terminalda /health javobida secure=true ekanini tekshiring. D3–D5 qo‘lda bajarilgan so‘rovlarni aynan o‘sha token va kiritmalar bilan qaytaring; natijalarni f01-after.txt, f02-after.txt, f03-after.txt nomlari bilan saqlang. Brauzer sahifasini yangilang.

```
curl --noproxy "*" -s "$U/health"
python3 audit.py > after.txt 2>&1
AFTER_RC=$?
cat after.txt
echo "$AFTER_RC"
diff -u before.txt after.txt
```

Kutiladigan natija: SUMMARY 12 / 12 va kod 0. diff farq topganda kod 1 qaytarishi tabiiy. T05:404; T07:200 va bo‘sh ro‘yxat; T08:kodlangan skript; T09:sarlavhalar. T02, T03 va T06 qonuniy foydalanish buzilmaganini tekshiradi.

Jurnal bilan dalilni bog‘lash

```
grep -F "BU_YERGA_REQUEST_ID" after-server.log
```

BU_YERGA_REQUEST_ID o‘rniga qo‘lda olingan bitta javobning X-Request-ID qiymatini qo‘ying. Logda mos path, status va secure=true bo‘lsin. Token va to‘liq query ataylab logga yozilmaydi. Bu minimal kuzatuv jurnali: SIEM, alerting yoki to‘liq hodisa tahlili imkoniyati emas.

Dalil D6. before/after to‘plami, chiqish kodlari, qo‘lda qayta sinov va log mosligi. Avtomatik T08 javob kodlanishini tekshiradi; brauzer JavaScript bajarmaganini mustaqil kuzatish D5da qoladi. 12/12 butun ilova xavfsizligini isbotlamaydi.

1.7. Xavfni baholash va qamrov jadvali

6-topshiriq. Har topilma uchun endpoint, kirish sharti, qayta hosil qilish qadamlari, kutilgan va amaldagi natija, dalil fayli, ildiz sabab, tuzatish hamda qayta sinov holatini yozing. Quyidagi soddalashtirilgan shkala faqat darsda ustuvorlik belgilash uchun; CVSS hisob-kitobi emas.

Ehtimollik E: 1 — maxsus shartlar kerak; 2 — qo‘shimcha imkoniyat kerak; 3 — oddiy mijoz so‘rovi yetarli. Ta’sir T: 1 — kichik cheklov; 2 — bitta foydalanuvchi kontekstiga ta’sir; 3 — foydalanuvchilar orasida maxfiylik yoki muhim amal buzilishi. Ball=E×T: 1–2 past, 3–4 o‘rta, 6–9 yuqori.

ID / topilma	E×T, namuna	Ustuvorlik / asos
F01 — IDOR	3×3=9	Yuqori: Alice Bob qaydini o‘qiydi
F02 — SQL inyeksiya	3×3=9	Yuqori: qidiruv egalar chegarasidan o‘tadi
F03 — reflected XSS	3×2=6	Yuqori: jabrlanuvchi URLni ochishi kerak
F04 — sarlavhalar yo‘q	2×1=2	Past: himoya qatlamining yetishmasligi

Bu baholar o‘quv ssenariysi namunasi. Laboratoriya ma’lumoti sun’iy; real tizimda aktiv qiymati, autentifikatsiya, foydalanish sharti va haqiqiy ta’sir bilan qayta baholanadi. XSS sinovi faqat lokal skript bajarilishini isbotlaydi, akkaunt egallanganini emas. Talaba boshqa ball bersa, dalil bilan asoslashi kerak.

OWASP Top 10:2025 bilan bog‘lash [1]

Toifa	Ushbu ishdagi qamrov
A01 — Broken Access Control	F01: qo‘lda tekshirildi va qayta sinaldi
A02 — Security Misconfiguration	F04: faqat ikki javob sarlavhasi tekshirildi
A03 — Software Supply Chain Failures	Tekshirilmagan; Python/OT inventari qayd etiladi
A04 — Cryptographic Failures	Tekshirilmagan; TLS va sirlar modeli yo‘q
A05 — Injection	F02 va F03: SQLi hamda reflected XSS
A06 — Insecure Design	Doira va egaga ruxsat talabi; to‘liq tahlil emas
A07 — Authentication Failures	Tekshirilmagan; demo token haqiqiy login emas
A08 — Software or Data Integrity Failures	Dalil xeshi olindi; reliz zanjiri tekshirilmagan
A09 — Security Logging & Alerting Failures	Log mosligi ko‘rildi; alerting tekshirilmagan
A10 — Mishandling of Exceptional Conditions	Noto‘g‘ri ID tekshirildi; keng qamrov yo‘q

Dalil D7. F01–F04 reyestri, asoslangan ball va qamrov. “Tekshirilmagan”, “topilmadi” va “tuzatildi” bir xil holat emas. ASVSdan kelgusidagi batafsil talablar tanlansa, versiya va aniq talab identifikatori bilan yozilsin. [2]

1.8. Xavfsiz ishlab chiqish va chiqarish qarori

7-topshiriq. Audit natijasini ishlab chiqish jarayoniga ulang. Xavfsizlik tekshiruvi reliz oxiridagi bir martalik ish bo‘lib qolmasin. NIST SSDFdagi xavfsiz ishlab chiqish yondashuvidan kelib chiqib, quyidagi amallarni loyiha rejasi sifatida belgilang. [6]

Bosqich	Aniq ish	Qabul dalili
Talab / dizayn	Qaydni faqat egasi o‘qishi; HTML kiritmasi matn bo‘lishi	Ruxsat matritsasi, D1
Kod yozish / ko‘rib chiqish	Parametrli SQL, obyekt egasi, kontekstli kodlash	get_note/search/preview izohi
Yig‘ish / sinash	audit.py va brauzer sinovi; versiyalar qaydi	12/12, D5 va D6
Chiqarish	Ochiq yuqori xavflar yo‘q; mas’ul tekshiruvi	F01–F04 holati va qaror
Ekspluatatsiya	Yangilash, log nazorati, yangi topilmani qayta sinash	Mas’ul, muddat, kuzatuv rejasi

Qaror yozish uchun namunaviy shakl

“Ushbu lokal o‘quv buildi uchun F01–F03 qayta sinovdan o‘tdi; F04 sarlavhalari mavjud. Avtomatik natija: __/12; brauzer dalili: __. Keyingi o‘quv bosqichiga: ruxsat / ruxsat yo‘q. Ochiq masala: __. Mas’ul: __; qayta tekshirish sanasi: __.”

Bu modelni ishlab chiqarishga chiqarish qarori doimo alohida qoladi: demo tokenlar, haqiqiy sessiya boshqaruvi yo‘qligi, HTTP, bir jarayonli o‘quv serveri va xotiradagi baza real xizmat talablari uchun yetarli emas. Python http.server ishlab chiqarish uchun tavsiya etilmaydi. [7]

Dalillarni saqlash

1-terminaldagi serverni Ctrl+C bilan to‘xtating. 2-terminalda quyidagilarni bir marta bajaring; f01–f03ning oldin/keyin fayllari avval yaratilgan bo‘lsin:

```
CASE_DIR="evidence-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -m 700 "$CASE_DIR"
cp app.py audit.py source.sha256 before.txt after.txt \
  before-server.log after-server.log f0*-*.txt "$CASE_DIR/"
(cd "$CASE_DIR" && sha256sum * > SHA256SUMS)
(cd "$CASE_DIR" && sha256sum -c SHA256SUMS)
chmod 400 "$CASE_DIR"/*
```

Har fayl uchun OK chiqishi kutiladi. Screenshotlar ham dalil katalogiga qo‘shilsa, xesh manifestini yangidan hisoblash kerak; eski manifestni tekshirilayotgan fayllar ichiga qo‘shmang. Xesh bayt mosligini tekshiradi, mustaqil imzo yoki vaqt tamg‘asi emas.

Dalil D8. Chiqarish qarori, SDLC rejasi va dalillar manifesti. Restart xotiradagi bazani dastlabki uch qaydga qaytaradi. Sinovlar faqat o‘qishdan iborat. Xavfsiz sozlamadan zaif rejimga qaytish odatiy rollback rejasi sifatida qabul qilinmaydi.

1.9. Vaqt, hisobot va baholash

Bosqich	Daqiqa	Natija
Doira, muhit, boshlang‘ich nazorat	10	D1–D2
IDOR va SQL inyeksiya	18	D3–D4
XSS va sarlavhalar	12	D5
Himoya va qayta sinov	18	D6
Risk, qamrov va SDLC qarori	12	D7–D8
Hisobotni yakunlash va himoya	10	Yakuniy xulosa
Jami	80	Tayyor VM va fayllar bilan

Topshiriladigan hisobot

6. Titul: fan, 5-semestr, A12, 1-variant, F.I.Sh., guruh va sana; muhit va audit doirasi.
7. D1–D8 dalillari: buyruq bilan natija birga ko‘rinsin. F01–F04 kartalarida oldin/keyin, sabab, tuzatish va qayta sinov holati bo‘lsin.
8. before.txt, after.txt, log mosligi, brauzer tasvirlari, xesh tekshiruvi va 5–7 gaplik xulosa. Tekshirilmagan yo‘nalishlar va chiqarish qarorini ajrating.

Baholash mezoni	Ball
Doira, arxitektura va muhitni to‘g‘ri qayd etish	1
IDOR: sabab, ta’sir va o‘z/begona obyekt nazorati	2
SQL inyeksiya: dalil va parametrli tuzatish izohi	2
XSS: brauzer dalili, kodlash va CSP farqi	2
Qayta sinov, funksional nazorat va log mosligi	1
Risk reyestri, qamrov, SDLC va chiqarish qarori	1
Hisobot, dalil xeshlari va og‘zaki himoya	1
Jami	10

Nazorat savollari

9. Nega parametrli SELECT mavjud bo‘lsa ham IDOR yuz berishi mumkin?
10. SQLi natijasida uchala yozuv chiqishiga qaysi mantiqiy shart sabab bo‘ldi?
11. curl orqali topilgan <script> bilan brauzerda bajarilgan skript dalili nimasi bilan farq qiladi?
12. 12/12 natijasi nega ASVSga to‘liq muvofiqlik yoki ishlab chiqarishga tayyorlik degani emas?

Qabul sharti. Faqat skrinshot yoki “hammasi PASS” yozuvi yetarli emas. Har muhim tuzatish uchun oldingi kamchilik, uning sababi, keyingi xavfsiz natija va qonuniy funksiyaning saqlanishi ko‘rsatiladi.

Muammolarni bartaraf etish va manbalar

Belgi	Tekshirish / yechim
Connection refused / port band	Server logini o‘qing; eski jarayonni o‘z terminalida Ctrl+C bilan to‘xtating.
SyntaxError / IndentationError	Ilova qismlarini tartib bilan qo‘shing; surilish va oddiy qo‘shtirnoqlarni tekshiring.
API doimo 401	X-Lab-Token sarlavhasi va demo-alice/demo-bob qiymatini tekshiring.
SQLi sinovi 400	Sinov satri va --data-urlencode qo‘shtirnoqlarini aynan ko‘chiring.
Himoyadan keyin ham 4 FAIL	Eski jarayon to‘xtaganmi? /health ichida secure=true ekanini tekshiring.
XSS tab nomi o‘zgarmadi	Zaif rejimmi? URL bitta satrmi? Brauzerda JavaScript yoqilganmi?
audit.py traceback; SUMMARY yo‘q	Bu sinov xatosi; tarmoq yoki fayl muammosini tuzatib, to‘liq to‘plamni qaytaring.
cp: cannot stat / xesh xatosi	Oldin/keyin dalil fayllarini yarating; manifestni aynan dalil katalogida tekshiring.

Kengaytirilgan topshiriq — vaqt qolsa

Faqat himoyalangan rejimda Bob nomidan Alice qaydiga murojaatni qo‘shing: 404 bo‘lsin. q=% bilan qidirilganda faqat token egasining qaydlari chiqsin. audit.pyga bitta qo‘shimcha mazmunli test kiriting va nega mavjud testlardan farq qilishini izohlang. Bu ish 80 daqiqalik minimumga kirmaydi.

Rasmiy manbalar

[1] OWASP Top 10:2025 — toifalar va qo‘llash doirasi

[2] OWASP ASVS — 5.0.0, tekshiruv talablari

[3] OWASP — Authorization Cheat Sheet

[4] OWASP — SQL Injection Prevention Cheat Sheet

[5] OWASP — Cross Site Scripting Prevention Cheat Sheet

[6] NIST SP 800-218 — SSDF Version 1.1

[7] Python — http.server, foydalanish chegaralari

Manbalar tekshirilgan sana: 07.09.2026. Mavzu taqdim etilgan fan dasturining A12 bandiga mos.

O‘qituvchi uchun. Dastlab app.py bayroqsiz ishga tushiriladi. Tayyorlashda ikkala rejimda jami 22 ta avtomatik audit HTTP so‘rovi tekshirildi: zaif holat 8/12, himoyalangan holat 12/12. Brauzer muhiti mavjud bo‘lmagani uchun tayyorlashda XSSning brauzerda bajarilishi tekshirilmadi; D5 darsda alohida bajariladi. O‘quv skripti to‘liq xavfsizlik skaneri emas.

1-ilova. app.py — A qism

A, B va C qismlarni ketma-ket ~/a12-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
import argparse, html, json, sqlite3, time, uuid
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlsplit, parse_qs

parser = argparse.ArgumentParser()
parser.add_argument('--secure', action='store_true')
SECURE = parser.parse_args().secure
TOKENS = {'demo-alice': 'alice', 'demo-bob': 'bob'}
DB = sqlite3.connect(':memory:')
DB.row_factory = sqlite3.Row
DB.execute('CREATE TABLE notes(id INTEGER, owner TEXT, title TEXT)')
DB.executemany('INSERT INTO notes VALUES(?,?,?)', [
    (1, 'alice', 'Alice plan'), (2, 'bob', 'Bob private'),
    (3, 'alice', 'Alice report')])

def get_note(owner, note_id):
    if SECURE:
        return DB.execute('SELECT * FROM notes WHERE id=? '
                          'AND owner=?', (note_id, owner)).fetchone()
    return DB.execute('SELECT * FROM notes WHERE id=?',
                      (note_id,)).fetchone()

def search(owner, term):
    if SECURE:
        return DB.execute('SELECT * FROM notes WHERE owner=? '
                          'AND title LIKE ?', (owner, '%' + term + '%'))
    sql = ("SELECT * FROM notes WHERE owner='" + owner +
           "' AND title LIKE '%" + term + "%'")
    return DB.execute(sql)

def preview(text):
    value = html.escape(text, quote=True) if SECURE else text
    return ('<!doctype html><meta charset="utf-8">'
            '<title>A12 Preview</title><h1>Preview</h1>'
            '<div>' + value + '</div>')
```

SECURE=False — zaif o‘quv tarmoqlari; --secure — tekshiriladigan tuzatishlar. Demo tokenlar haqiqiy autentifikatsiya mexanizmi emas. Baza har ishga tushganda xotirada yangidan yaratiladi.

1-ilova. app.py — B qism

A, B va C qismlarni ketma-ket ~/a12-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass

    def reply(self, status, obj, is_html=False):
        rid = uuid.uuid4().hex
        body = (obj if is_html else json.dumps(obj)).encode('utf-8')
        self.send_response(status)
        self.send_header('Content-Type',
                         'text/html; charset=utf-8' if is_html
                         else 'application/json; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.send_header('Cache-Control', 'no-store')
        self.send_header('X-Request-ID', rid)
        if SECURE:
            self.send_header('X-Content-Type-Options', 'nosniff')
            self.send_header('Content-Security-Policy',
                             "default-src 'none'; frame-ancestors 'none'; "
                             "base-uri 'none'; form-action 'none'")
        self.end_headers()
        self.wfile.write(body)
        print(json.dumps({'ts': time.time(), 'request_id': rid,
                          'path': urlsplit(self.path).path,
                          'status': status, 'secure': SECURE}), flush=True)

    def do_GET(self):
        try:
            path = urlsplit(self.path).path
            qs = parse_qs(urlsplit(self.path).query,
                          keep_blank_values=True, max_num_fields=10)
            if path == '/health':
                return self.reply(200, {'ok': True, 'secure': SECURE})
            if path == '/preview':
                text = qs.get('text', [''])[0]
```

reply javob identifikatori va minimal hodisa satrini hosil qiladi. Token, query va foydalanuvchi yuborgan matn logga nusxalanmaydi. CSP va nosniff faqat himoyalangan rejimda qo‘shiladi.

1-ilova. app.py — C qism

A, B va C qismlarni ketma-ket ~/a12-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
                if len(text) > 500:
                    return self.reply(400, {'error': 'bad_input'})
                return self.reply(200, preview(text), True)
            if path not in ('/api/note', '/api/search'):
                return self.reply(404, {'error': 'not_found'})
            owner = TOKENS.get(self.headers.get('X-Lab-Token'))
            if owner is None:
                return self.reply(401, {'error': 'unauthorized'})
            if path == '/api/note':
                note_id = qs.get('id', [''])[0]
                if not note_id.isascii() or not note_id.isdecimal():
                    return self.reply(400, {'error': 'bad_input'})
                row = get_note(owner, note_id)
                if row is None:
                    return self.reply(404, {'error': 'not_found'})
                return self.reply(200, dict(row))
            term = qs.get('q', [''])[0]
            if len(term) > 100:
                return self.reply(400, {'error': 'bad_input'})
            rows = [dict(r) for r in search(owner, term)]
            return self.reply(200, {'notes': rows})
        except (ValueError, sqlite3.Error):
            return self.reply(400, {'error': 'bad_input'})

print('A12 127.0.0.1:8012 secure=' + str(SECURE), flush=True)
server = HTTPServer(('127.0.0.1', 8012), Handler)
server.timeout = 2
try:
    server.serve_forever()
except KeyboardInterrupt:
    pass
finally:
    server.server_close()
    DB.close()
```

Faqat belgilangan GET yo‘llari xizmat ko‘rsatadi; diskdagi fayllar HTTP orqali tarqatilmaydi. Bu kod katta yuklama, xizmat uzluksizligi va production xavfsizligi uchun mo‘ljallanmagan.

2-ilova. audit.py — A qism

A va B qismlarni ketma-ket ~/a12-lab/audit.py fayliga birlashtiring. Ishga tushirish: python3 audit.py. Bu fayl app.py ishlayotgan vaqtda ikkinchi terminalda bajariladi.

```
import html, json, sys
from urllib.request import Request, build_opener, ProxyHandler
from urllib.error import HTTPError
from urllib.parse import urlencode

BASE = 'http://127.0.0.1:8012'
client = build_opener(ProxyHandler({}))
results = []

def get(path, token='demo-alice'):
    headers = {'X-Lab-Token': token} if token else {}
    try:
        response = client.open(Request(BASE + path, headers=headers),
                               timeout=5)
    except HTTPError as error:
        response = error
    with response:
        return response.code, response.headers, response.read().decode()

def check(name, passed):
    results.append(bool(passed))
    print(('PASS ' if passed else 'FAIL ') + name)

s, h, b = get('/health')
check('T01 health', s == 200 and json.loads(b)['ok'])
s, h, b = get('/api/note?id=1')
check('T02 own_note', s == 200 and json.loads(b)['owner'] == 'alice')
s, h, b = get('/api/note?id=2', 'demo-bob')
check('T03 bob_own_note', s == 200 and json.loads(b)['owner'] == 'bob')
s, h, b = get('/api/note?id=1', None)
```

Har tekshiruv PASS yoki FAIL chiqaradi. HTTP xatolari ham tekshiriladigan javob sifatida olinadi; ulanish yoki noto‘g‘ri JSON kabi kutilmagan xatolar to‘plamni to‘xtatadi. To‘liq SUMMARY bo‘lmasa natija qabul qilinmaydi.

2-ilova. audit.py — B qism

A va B qismlarni ketma-ket ~/a12-lab/audit.py fayliga birlashtiring. Ishga tushirish: python3 audit.py. Bu fayl app.py ishlayotgan vaqtda ikkinchi terminalda bajariladi.

```
check('T04 no_token', s == 401)
s, h, b = get('/api/note?id=2')
check('T05 foreign_note', s == 404 and 'Bob private' not in b)
s, h, b = get('/api/search?' + urlencode({'q': 'Alice'}))
check('T06 normal_search', s == 200 and
      [n['id'] for n in json.loads(b)['notes']] == [1, 3])
s, h, b = get('/api/search?' + urlencode({'q': "' OR 1=1-- "}))
check('T07 sql_input', s == 200 and json.loads(b)['notes'] == [])
payload = '<script>document.title="A12-XSS"</script>'
s, h, b = get('/preview?' + urlencode({'text': payload}))
check('T08 output_encoding', s == 200 and html.escape(payload) in b
      and payload not in b)
check('T09 headers', h.get('X-Content-Type-Options') == 'nosniff'
      and h.get('Content-Security-Policy') ==
      "default-src 'none'; frame-ancestors 'none'; "
      "base-uri 'none'; form-action 'none'")
s, h, b = get('/api/note?id=xyz')
check('T10 invalid_id', s == 400 and 'Traceback' not in b)
s, h, b = get('/api/note?id=999')
check('T11 missing_note', s == 404)
s, h, b = get('/preview?' + urlencode({'text': 'Salom & <kitob>'}))
check('T12 normal_text', s == 200 and 'Salom' in b)
print('SUMMARY', sum(results), '/', len(results))
sys.exit(0 if all(results) else 1)
```

Natija: zaif rejimda T05, T07, T08, T09 — FAIL va kod 1; himoyalangan rejimda 12/12 va kod 0. T12 oddiy matn javobini tekshiradi; to‘liq foydalanuvchi interfeysi regressiyasi emas.